2 академических часа

Установка Docker и управление контейнерами

Лекция №2 · Docker и контейнеризация приложений

Цель лекции

- Устанавливать Docker Engine из официального репозитория Debian.
- Управлять службой Docker средствами systemd.
- Создавать и контролировать контейнеры через CLI.
- Диагностировать ошибки запуска и безопасно удалять учебные контейнеры.

Вспомним лекцию №1

- Docker Client отправляет команды Docker Daemon.
- Docker Image — шаблон; Container — экземпляр образа.
- Контейнер существует как управляемая среда выполнения процесса.
- В Linux Docker Engine обычно работает как служба.

Учебные вопросы

№	Тема	Время
1	Подготовка Debian и способы установки	15 мин
2	Установка Docker Engine через APT	20 мин
3	Управление Docker Daemon и права	15 мин
4	Создание и запуск контейнеров	20 мин
5	Жизненный цикл и диагностика	15 мин
6	Итоги и контроль	5 мин

Подготовка системы

Почему важно выбрать правильный способ установки

Целевая учебная среда

- Debian 13 (Trixie) внутри VirtualBox либо на отдельном компьютере.
- Пользователь с правом выполнять команды через sudo.
- Рабочее подключение к интернету и репозиториям Debian.
- Для внешнего доступа к контейнерам заранее проверяют сетевые настройки ВМ.

Проверяем версию Debian

BASH

```
cat /etc/os-release
dpkg --print-architecture
```

Что важно

- VERSION_CODENAME должен содержать trixie для Debian 13.
- Команда dpkg показывает архитектуру пакетов: например amd64.
- Не копируйте инструкции другой ОС без проверки её версии.

Проверяем права и сеть

```bash
whoami
sudo -v
getent hosts download.docker.com
```

Что важно

- sudo -v проверяет возможность выполнения команд администратора.
- getent hosts проверяет DNS-разрешение имени репозитория.
- Успешное разрешение DNS ещё не гарантирует доступ к HTTPS.

Способы установки Docker

- Официальный APT-репозиторий Docker: рекомендуемый способ для повторяемой установки и обновлений.
- Установка DEB-пакетов вручную: для ограниченных или отдельных сред.
- Скрипт быстрого развёртывания: предназначен прежде всего для тестовых сред.
- Пакет docker.io из репозитория Debian и пакет docker-ce из Docker Repository — разные способы поставки.

Конфликтующие пакеты: не удаляем вслепую

- Перед установкой проверить наличие docker.io, docker-compose, podman-docker, containerd и runc.
- Если обнаружен работающий Docker — сначала выяснить, какие контейнеры и данные используются.
- Удаление пакета не равно удалению образов и томов, но может нарушить работу существующих сервисов.
- На учебной новой ВМ рекомендована чистая исходная система.

Проверяем существующую установку

```bash
command -v docker || true
dpkg -l docker.io docker-ce containerd runc 2>/dev/nu
```

Что важно

- Отсутствие docker означает, что CLI не найден в PATH.
- Список dpkg покажет установленные или известные пакеты.
- В учебной чистой ВМ конфликты обычно отсутствуют, но проверка обязательна.

Установка Docker Engine

Подключение официального APT-репозитория

Установка Docker в Debian

Пошаговая схема установки Docker Engine из официального APT-репозитория

1 Подготовка системы



- Обновить список пакетов
- Установить ca-certificates, curl, gnupg

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
```

2 Добавить ключ Docker



- Создать каталог /etc/apt/keyrings
- Скачать GPG-ключ

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

3 Подключить репозиторий



- Использовать официальный репозиторий Docker
- Архитектура и версия Debian определяются автоматически

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian $(. /etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
```

4 Установить Docker Engine



- Установить Docker Engine, CLI и плагины
- Compose устанавливается как plugin

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

5 Проверить службу



- Проверить статус systemd
- Убедиться, что Docker запущен

```
sudo systemctl status docker
sudo docker version
sudo docker info
```

Служба: `dockerd`

6 Тестовый запуск



- Запустить hello-world
- Проверить, что образ скачан и контейнер выполнился

```
sudo docker run hello-world
```

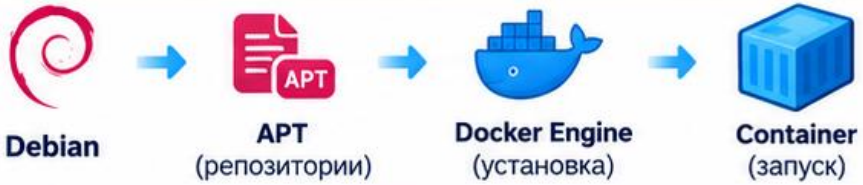
В выводе должно появиться:

`Hello from Docker!`

Что важно помнить

1. Используйте официальный репозиторий Docker.
2. На занятиях команды выполняем через `sudo`.
3. Группа `docker` даёт почти root-доступ.
4. Для проверки удобно использовать `hello-world` и `nginx`.

Краткая схема



Этапы установки

- 1. Установить ca-certificates и curl.
- 2. Добавить ключ подписи официального репозитория в /etc/apt/keyrings.
- 3. Создать /etc/apt/sources.list.d/docker.sources.
- 4. Обновить индексы пакетов и установить компоненты Engine.
- 5. Проверить службу и тестовый контейнер.

Обновляем индексы и устанавливаем зависимости

BASH

```
sudo apt update
sudo apt install -y ca-certificates curl
```

Что важно

- apt update получает сведения о доступных пакетах.
- ca-certificates нужен для проверки TLS-сертификатов; curl — для загрузки ключа.
- Установка успешна, если apt завершился без ошибок.

Добавляем ключ репозитория

BASH

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/deb
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Что важно

- Ключ сохраняется как docker.asc; это не исполняемый скрипт.
- Параметр -fsSL делает загрузку удобнее для сценариев установки.
- chmod a+r позволяет APT прочитать ключ подписи.

Создаём docker.sources

```bash
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Что важно

- APT использует формат deb822 для описания источника.
- Suites определяется из /etc/os-release: для Debian 13 — trixie.
- Файл docker.sources хранит адрес репозитория и путь к ключу.

Обновляем список пакетов

BASH

```
sudo apt update
apt-cache policy docker-ce
```

Что важно

- После подключения репозитория apt должен находить docker-ce.
- apt-cache policy показывает установленную и доступную кандидатную версию.
- Если Candidate отсутствует — проверить DNS, ключ и docker.sources.

Устанавливаем компоненты

```bash
sudo apt install -y docker-ce docker-ce-cli containe
```

Что важно

- docker-ce: Docker Engine и служба dockerd.
- docker-ce-cli: команда docker.
- containerd.io: компоненты выполнения контейнеров.
- buildx и compose: плагины сборки и многоконтейнерных приложений.

Пакеты официальной установки

Пакет	Назначение
docker-ce	Docker Engine (daemon и сопутствующие компоненты)
docker-ce-cli	Клиент docker
containerd.io	Контейнерный runtime и зависимости
docker-buildx-plugin	Современные возможности сборки
docker-compose-plugin	Команда docker compose

Проверяем установку

BASH

```
docker --version
sudo docker version
sudo docker compose version
```

Что важно

- docker --version показывает установленный клиент CLI.
- docker version выводит сведения о Client и Server, когда daemon доступен.
- Compose установлен как плагин: синтаксис docker compose.

Тест после установки

```bash
sudo docker run --rm hello-world
```

Что важно

- Docker загружает образ при отсутствии локальной копии и создаёт контейнер.
- Программа выводит Hello from Docker! и завершается.
- --rm удаляет контейнер после завершения; образ остаётся.

Служба Docker и права доступа

systemd, dockerd, сокет и безопасность

Архитектура службы Docker

Как Docker CLI, Docker Daemon и контейнерные компоненты взаимодействуют в Linux

1. Слой пользователя

Пользователь



Примеры команд:

```
docker ps
docker run nginx
docker images
```

2. Слой клиента (CLI)

Docker CLI

Клиент командной строки

Unix socket / Docker API

3. Слой службы (Docker Engine)

Docker Daemon (dockerd)

Системная служба

- Принимает команды от клиента
- Управляет образами, контейнерами, сетями и томами
- Работает через Docker API
- Запускается как служба systemd

pull / push

Загрузка и публикация Docker-образов

Docker Registry / Docker Hub

Хранилище Docker-образов (публичное)

Локальные объекты Docker

- **Images** — Docker-образы
- **Networks** — Сети
- **Volumes** — Тома данных
- **Containers** — Контейнеры

4. Слой выполнения (runtime)

containerd

Управление жизненным циклом контейнеров

runc

Низкоуровневый запуск контейнера

5. Слой контейнеров

Контейнеры (изолированные процессы)

- nginx
- api
- db

Как всё работает? (последовательность шагов)

1. **Пользователь вводит команду**
   Например, docker run nginx
2. **CLI отправляет запрос**
   Через Unix socket (Docker API)
3. **dockerd обрабатывает запрос**
   Управляет образами, контейнерами, сетями и томами
4. **containerd и runc запускают контейнер**
   Создают и запускают контейнер
5. **Контейнер выполняет приложение**
   Например, nginx, api или db в изолированном окружении

Ключевые выводы:

- **CLI — интерфейс команд**
  Позволяет пользователю управлять Docker.
- **dockerd — основная служба Docker**
  Обрабатывает команды и управляет всеми объектами.
- **containerd и runc — внутренние компоненты запуска**
  Отвечают за создание и запуск контейнеров.
- **Контейнеры — изолированные процессы**
  Запускают приложения в собственном окружении, на общей Linux-системе.

Docker как системная служба

- В большинстве установок Debian Docker управляется сервисом docker.service.
- Основной daemon — процесс dockerd; runtime-компоненты выполняют контейнеры.
- Доступ CLI к daemon обычно осуществляется через Unix socket /var/run/docker.sock.
- Состояние сервиса нельзя путать с состоянием отдельных контейнеров.

Состояние службы

BASH

```
sudo systemctl status docker --no-pager
sudo systemctl is-active docker
sudo systemctl is-enabled docker
```

Что важно

- active означает, что служба работает.
- enabled означает настройку запуска при старте системы.
- Состояние конкретного контейнера проверяют отдельной командой docker ps.

Управление службой

BASH

```
sudo systemctl stop docker
sudo systemctl start docker
sudo systemctl restart docker
```

Что важно

- stop останавливает службу; действия могут повлиять на контейнеры.
- start запускает службу.
- restart перезапускает службу; только в учебной ВМ, не на общем сервере.

Автозапуск Docker

```bash
sudo systemctl enable docker
sudo systemctl is-enabled docker
```

Что важно

- enable создаёт настройку автоматического старта при загрузке.
- Не путайте автозапуск Docker Daemon с политикой запуска контейнеров.
- Если служба была запущена, enable не требует её перезапуска.

Диагностика systemd

BASH

```
sudo journalctl -u docker -n 40 --no-pager
```

Что важно

- journalctl показывает журнальные записи именно docker.service.
- -n 40 ограничивает вывод последними 40 строками.
- Ошибки запуска Engine ищут здесь и в состоянии службы.

Почему обычно используется sudo?

- Unix socket Docker обычно доступен root и пользователям группы docker.
- Членство в docker группе даёт практически root-уровень полномочий.
- На учебных ВМ безопаснее вводить sudo docker, не меняя групповые права.
- Rootless Docker — отдельный режим, который в данном занятии не настраивается.

Ошибки прав доступа

Симптом	Причина / действие
permission denied для docker.sock	Недостаточные права: использовать sudo; не раздавать доступ без политики
Cannot connect to the Docker daemon	Проверить systemctl status docker и корректный Docker context
command not found	Проверить установку docker-ce-cli и PATH
Temporary failure resolving	Проверить DNS и доступ к репозиторию

Создание и запуск контейнеров

docker run, параметры и режимы

Команда docker run

- Объединяет создание контейнера и запуск его основного процесса.
- При отсутствии образа Docker обычно пытается получить его из реестра.
- Опции слева от имени образа задают режим, имя и порты.
- Команда после образа может переопределять стандартную команду запуска.

Запуск интерактивного контейнера

BASH

```
sudo docker run --rm -it debian:13 bash
```

Что важно

- -i оставляет стандартный ввод открытым; -t создаёт псевдотерминал.
- bash запускается внутри контейнера Debian.
- exit завершает оболочку; --rm удалит остановленный контейнер.

Запуск контейнера в фоне

BASH

```
sudo docker run -d --name web2 -p 8080:80 nginx:stabl
```

Что важно

- -d: detached, фоновый режим.
- --name задаёт удобное имя, которое должно быть уникальным.
- -p 8080:80 публикует порт 80 контейнера через порт 8080 хоста.
- Выполните curl -I http://localhost:8080 для проверки HTTP.

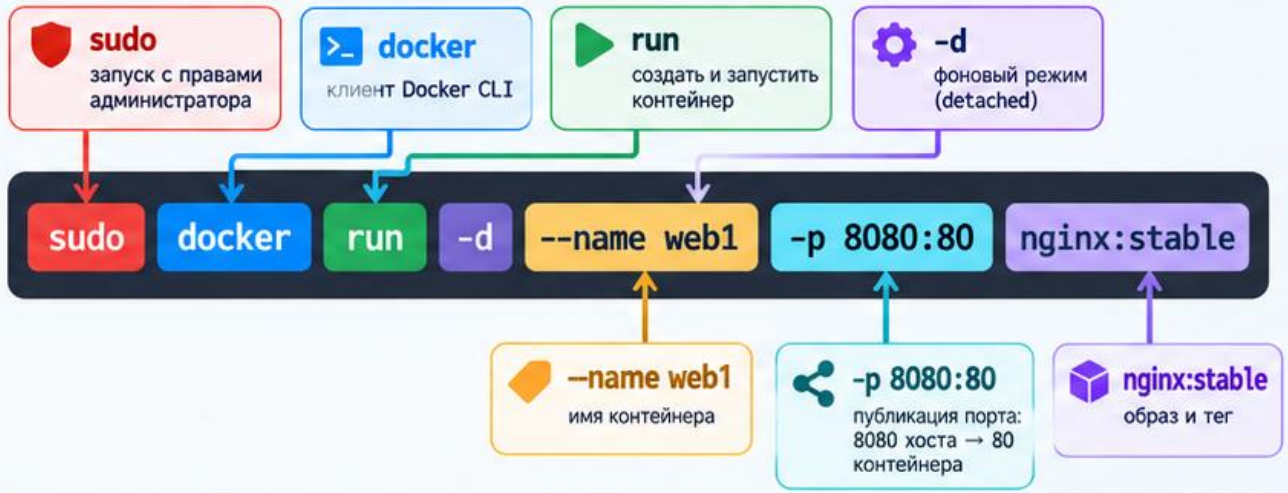
Анатомия docker run

Как одна команда создаёт и запускает контейнер

Что делает docker run

1. Проверяет, есть ли образ локально.
2. При необходимости скачивает образ из реестра.
3. Создаёт контейнер из образа.
4. Применяет параметры запуска.
5. Запускает основной процесс контейнера.

Разбор команды по частям



Как это работает под капотом



Часто используемые параметры

Параметр	Описание
-d	запуск в фоне
-it	интерактивный терминал
--name	задать имя контейнера
-p	проброс портов
-e	переменные окружения
-v	тома и каталоги
--rm	удалить контейнер после завершения
--restart	политика перезапуска

Что получаем в результате

- **Контейнер создан** — Из образа nginx:stable
- **Контейнер запущен** — Работает в фоновом режиме (-d)
- **Сервис доступен на порту 8080** — Порт 8080 хоста → 80 контейнера
- **Проверка:** `http://localhost:8080`

Важно помнить

1. Образ — шаблон, контейнер — его экземпляр
2. `docker run` объединяет `create + start`
3. Если основной процесс завершится, контейнер остановится

Параметры docker run

Параметр	Назначение
--name	Имя контейнера
-d	Запуск в фоне
-it	Интерактивный терминал
--rm	Автоматическое удаление после завершения
-p 8080:80	Публикация порта хоста на порт контейнера
--restart unless-stopped	Политика перезапуска

Публикация портов и безопасность

- Формат -p HOST_PORT:CONTAINER_PORT.
- Без указания адреса Docker может слушать на всех доступных интерфейсах хоста.
- Для проверки только с Debian используйте -p 127.0.0.1:8080:80.
- Для доступа из Windows через VirtualBox нужны правильные настройки сети ВМ.

Локальное опубликование порта

BASH

```
sudo docker run -d --name web-local -p 127.0.0.1:8081
curl -I http://127.0.0.1:8081
```

Что важно

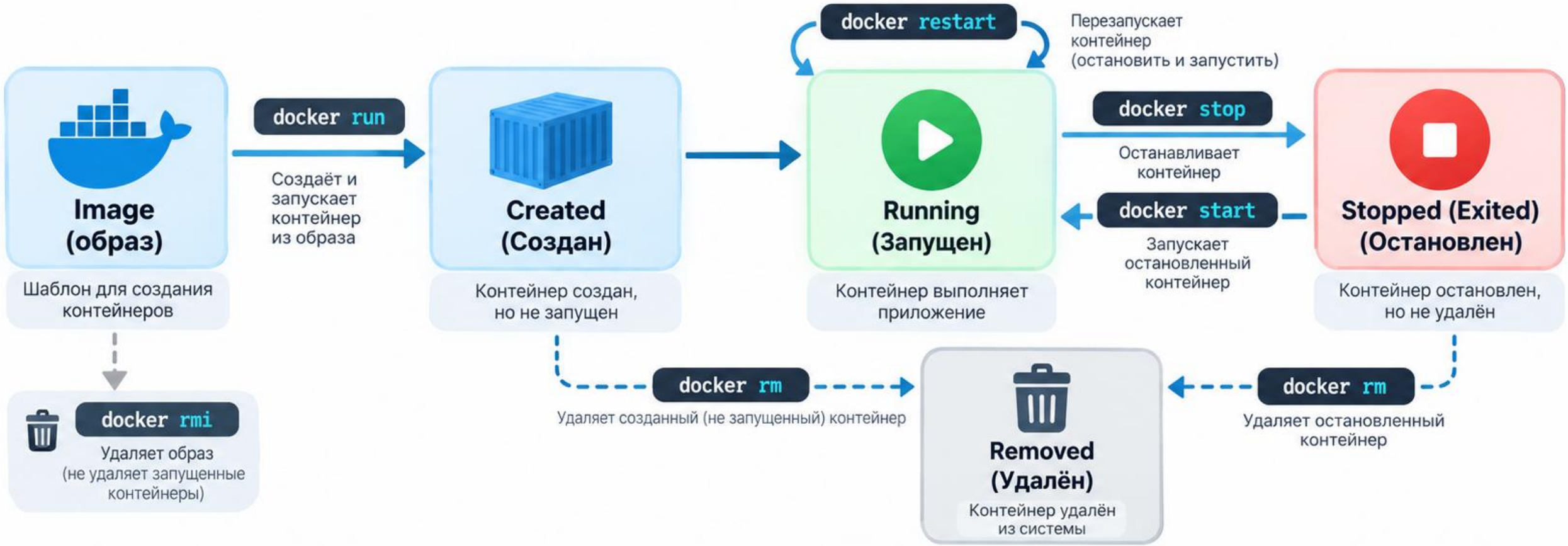
- Привязка к 127.0.0.1 ограничивает доступ loopback-интерфейсом хоста.
- В выводе curl ожидается успешный HTTP-ответ при исправной работе Nginx.
- На Debian без curl его необходимо предварительно установить.

Жизненный цикл и диагностика

Статусы контейнеров и управляющие команды

Жизненный цикл контейнера

Основные состояния Docker-контейнера и команды управления



Важно помнить

- Контейнер создается из образа.
- После stop контейнер не удаляется.
- Один образ может создать много контейнеров.
- Удаление контейнера не удаляет образ автоматически.

Что обычно видит студент

`docker ps`	— только запущенные контейнеры
`docker ps -a`	— все контейнеры
`docker logs <имя>`	— журналы контейнера
`docker exec -it <имя> /bin/sh`	— вход внутрь контейнера

Что означает состояние контейнера?

- Created — контейнер создан, но основной процесс не был запущен.
- Running — основной процесс выполняется.
- Exited — основной процесс завершился; контейнер сохранён.
- Paused и Restarting — дополнительные состояния; изучаются обзорно.

Смотрим контейнеры

```bash
sudo docker ps
sudo docker ps -a
```

Что важно

- docker ps показывает работающие контейнеры.
- docker ps -a выводит также остановленные и созданные контейнеры.
- Сравните ID, NAMES, STATUS, PORTS.

Управляем ранее созданным контейнером

BASH

```
sudo docker stop web2
sudo docker start web2
sudo docker restart web2
```

Что важно

- stop отправляет основной программе сигнал завершения и при необходимости принудительно останавливает её.
- start запускает ранее созданный контейнер.
- restart останавливает и запускает контейнер заново.

Получаем журналы

BASH

```
sudo docker logs web2
sudo docker logs --tail 20 web2
```

Что важно

- logs читает сообщения контейнера из доступного logging driver.
- --tail 20 ограничивает вывод последними 20 строками.
- Для демонстрации откройте страницу Nginx перед повторным вызовом logs.

Выполняем команду внутри

BASH

```
sudo docker exec web2 hostname
sudo docker exec -it web2 /bin/sh
```

Что важно

- exec запускает дополнительную команду в существующем работающем контейнере.
- hostname показывает имя узла внутри контейнера.
- /bin/sh открывает оболочку; exit завершает только оболочку, не контейнер.

Изучаем конфигурацию

BASH

```
sudo docker inspect web2
sudo docker port web2
```

Что важно

- inspect выводит подробные параметры и текущее состояние.
- docker port показывает опубликованные порты контейнера.
- Используйте эти команды при ошибках доступа к веб-серверу.

Удаление контейнеров

BASH

```
sudo docker stop web2
sudo docker rm web2
```

Что важно

- Остановите контейнер перед обычным удалением.
- rm удаляет экземпляр контейнера и его записываемый слой, но не образ.
- Не применяйте массовые prune-команды на общих учебных хостах.

Типовые ошибки и алгоритм диагностики

- Служба не работает: systemctl status docker, journalctl -u docker.
- Контейнер завершился: docker ps -a, docker logs NAME.
- Контейнер не создаётся: проверить имя, образ и сообщение Docker.
- Нет доступа к Nginx: docker ps, docker port, curl и настройки VirtualBox.
- Порт занят: выбрать иной свободный порт хоста.

Диагностика: сайт недоступен

1	**Шаг 1**	Проверьте, что daemon работает: sudo systemctl is-active docker.
2	**Шаг 2**	Проверьте контейнер: sudo docker ps -a.
3	**Шаг 3**	Если Exited — прочитайте sudo docker logs web2.
4	**Шаг 4**	Если Running — проверьте sudo docker port web2.
5	**Шаг 5**	Выполните curl -I http://localhost:8080 из Debian.
6	**Шаг 6**	Если запрос с Windows не проходит — отдельно проверьте сеть VirtualBox.

Разница между остановкой Docker и контейнера

- docker stop web2 действует на один контейнер.
- systemctl stop docker действует на системную службу Engine.
- Поведение уже работающих контейнеров при остановке daemon зависит от конфигурации, например live-restore.
- Не останавливайте dockerd на сервере с чужими приложениями ради эксперимента.

Итоги лекции

1. Официальная установка Debian 13 выполняется через docker.sources и пакеты docker-ce.
2. systemctl управляет службой Docker, а docker CLI — объектами Docker Engine.
3. Команда docker run создаёт и запускает контейнер; -d, -it, --name и -p меняют его параметры.
4. docker ps -a, logs, exec, inspect помогают диагностировать неисправности.
5. Контейнеры нужно удалять адресно, сохраняя данные других проектов.

Контрольные вопросы

1. Чем docker-ce отличается от docker-ce-cli?
2. Зачем Docker нужен отдельный APT-репозиторий и ключ подписи?
3. Что делает systemctl enable docker?
4. Почему docker без sudo может выдавать permission denied?
5. Чем отличается docker run от docker start?
6. В чём разница между -d и -it?
7. Как интерпретировать -p 127.0.0.1:8080:80?
8. Почему docker ps может быть пустым, хотя docker ps -a показывает контейнеры?
9. Как найти причину остановки контейнера?
10. Что останется после docker rm?